CS 6515: Graduate Algorithms

Lecture 3: From Divide-and-Conquer to Dynamic Programming

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. HW 1 is due this Thursday (Sep 3).
2. Office Hours schedule available on Piazza.
3. Plan for today: Finish median finding in linear time and discussion on Divide-and-conquer. Start Dynamic Programming and discuss Largest Contiguous Sum and Longest Common Subsequence problems.

2 Linear Time Median finding

Input. Distinct numbers $A[0], \ldots, A[n-1]$ and a rank $k \in \{1, 2, \ldots, n\}$.

Goal. Find the k-th smallest element. It has exactly $k-1$ smaller elements and $n-k$ larger elements.

Sorting first takes $\Theta(n \log n)$ time. The selection algorithm of Blum, Floyd, Pratt, Rivest, and Tarjan achieves deterministic linear time by finding a provably good pivot.

Theorem (deterministic selection). The k-th order statistic of n numbers can be found in $\mathcal{O}(n)$ time.

2.1 Proof plan: what makes a pivot good?

Suppose we choose a pivot p and partition the input into

$$L = \{x : x < p\}, \qquad \{p\}, \qquad G = \{x : x > p\}.$$

Once we know $|L|$, we can determine whether the desired element is p, lies in L, or lies in G. We then recurse on only the relevant side.

An arbitrary pivot need not help: if it is the smallest or largest element, the recursive problem can still have size $n-1$. Repeated linear-time partitioning would then lead to the recurrence

$$T(n) \leq T(n-1) + cn,$$

which permits quadratic running time. We therefore need a pivot that is bounded away from both extremes; it need not be the exact median.

Definition (good pivot). We call p a *good pivot* if both L and G have size at most

$$\frac{7n}{10} + \mathcal{O}(1).$$

Equivalently, up to constant rounding terms, at least $3n/10$ elements lie on each side of p and can be discarded when we recurse on the other side.

Our proof will follow three steps:

1. In $\mathcal{O}(n)$ nonrecursive work, form about $n/5$ representative elements, then recursively select their median. This costs $T(\lceil n/5 \rceil) + \mathcal{O}(n)$ and produces the pivot p.
2. Prove that this particular p is a good pivot.
3. Partition around p in $\mathcal{O}(n)$ time and recurse on at most $7n/10 + \mathcal{O}(1)$ elements.

Consequently, the running time will satisfy the recurrence previewed in Section 2:

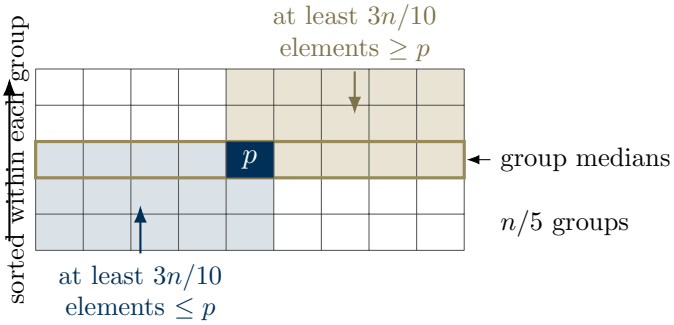
$$T(n) \leq T(\lceil n/5 \rceil) + T(\lceil 7n/10 \rceil) + cn = \mathcal{O}(n).$$

The construction below is designed precisely to establish the missing good-pivot guarantee.

2.2 Finding a good pivot

For clarity, first suppose n is divisible by 5 and ignore harmless rounding terms.

1. Partition the input into $n/5$ groups of five.
2. Sort each group and record its median. Because every group has constant size, this takes $\mathcal{O}(n)$ total time.
3. Recursively find the median p of the $n/5$ group medians.



Columns are shown ordered by their medians only to visualize the counting argument.

Claim. The median-of-medians pivot p has at least $3n/10$ elements no smaller than it and at least $3n/10$ elements no larger than it, up to constant rounding terms.

Proof of the claim. At least half of the group medians are at least p. In each corresponding group, the median and the two larger elements are all at least p. Hence at least

$$3 \cdot \frac{1}{2} \cdot \frac{n}{5} = \frac{3n}{10}$$

elements are at least p. The symmetric argument gives at least $3n/10$ elements at most p. □

2.3 Discarding one side

Partition the input in $\mathcal{O}(n)$ time into

$$L = \{x : x < p\}, \qquad \{p\}, \qquad G = \{x : x > p\}.$$

Then:

- if $k \le |L|$, recurse on L with rank k;
- if $k = |L| + 1$, return p;
- if $k > |L| + 1$, recurse on G with rank $k - |L| - 1$.

Because p is a good pivot, the recursive side contains at most $7n/10 + \mathcal{O}(1)$ elements.

2.4 Running time

The recursive call that finds the pivot uses an input of size about $n/5$, and the recursive selection call uses an input of size at most $7n/10 + \mathcal{O}(1)$. Grouping, sorting the five-element groups, and partitioning cost $\mathcal{O}(n)$. Thus, ignoring rounding terms,

$$T(n) \le T(n/5) + T(7n/10) + cn.$$

Exercise. Include the rounding terms and match this recurrence to the selection-type theorem from Section 2. Conclude that deterministic selection runs in $\mathcal{O}(n)$ time.

Divide-and-conquer takeaway. We divide an instance into smaller, largely separate subproblems, solve them recursively, and combine their answers. The quality of the split and the cost of the combine step determine the recurrence. Merge sort, maximum difference, and deterministic selection illustrate three different recurrence shapes.

3 Dynamic programming

Dynamic programming (DP) is useful when a problem has many smaller *overlapping* subproblems. Instead of resolving the same subproblem many times, we store its answer and reuse it.

Most DP algorithms follow four design steps:

1. define a collection of subproblems, often index by non-negative integers $i, j, \ldots$;
2. give a recurrence relating each subproblem to smaller ones, i.e., if we have solutions to the subproblems (like stored in an array $M[i, j, \ldots]$) then we can solve the original problem;
3. identify base cases and an evaluation order;
4. recover the desired solution, not merely its value, if needed.

4 Largest contiguous sum

Input. An array $A[0], A[1], \ldots, A[n-1]$ of numbers (possibly negative). Again assume that each number uses $\mathcal{O}(1)$ bits.

Goal. Find an interval $[x, y]$ maximizing

$$\sum_{i=x}^{y} A[i].$$

We permit the empty interval, whose value is 0.

Example. For

$$A = [5, 15, -30, 10, -5, 40, 10],$$

the interval $[10, -5, 40, 10]$ has value 55 and is optimal.

The key is to define a subproblem that remembers enough information to extend a solution by one position.

Definition (subproblem). For $0 \le i < n$, let $M[i]$ be the largest sum of a contiguous subarray that ends exactly at position i.

Any nonempty interval ending at i either begins at i, or extends the best interval ending at $i-1$. Therefore

$$M[0] = A[0], \qquad M[i] = \max\{A[i],\ A[i] + M[i-1]\} \quad (i \ge 1).$$

The desired value is

$$\max\left\{0, \max_{0 \le i < n} M[i]\right\}.$$

There are n states and each takes constant time, so the algorithm runs in $\mathcal{O}(n)$ time. Only the preceding value is needed if we want the optimum value alone, giving $\mathcal{O}(1)$ auxiliary space.

Exercise. Fill in the values $M[i]$ for the example. Then augment the algorithm so that it returns the two endpoints of an optimal interval.

Correctness invariant. After computing $M[i]$, it equals the best sum among all intervals ending at i. This follows by induction: removing $A[i]$ from an interval that starts before i leaves an interval ending at $i-1$, and the recurrence chooses the better of extending that interval and starting anew.

5 Longest common subsequence

Input. Sequences

$$A = a_1 a_2 \cdots a_n, \qquad B = b_1 b_2 \cdots b_m.$$

A *subsequence* is obtained by deleting zero or more elements without changing the order of those retained. Unlike a substring, its elements need not occupy consecutive positions.

Goal. Find the maximum length of a sequence that is a subsequence of both A and B.

Example. If $A = \texttt{ABCDEFG}$ and $B = \texttt{BCDGKAB}$, then `BCDG` is a common subsequence of length 4.

Definition (prefix subproblem). For $0 \le i \le n$ and $0 \le j \le m$, let $D[i, j]$ be the length of a longest common subsequence of $a_1 \cdots a_i$ and $b_1 \cdots b_j$.

The empty prefix gives the base cases

$$D[0, j] = D[i, 0] = 0.$$

For $i, j \ge 1$,

$$D[i, j] = \begin{cases} D[i-1, j-1] + 1, & a_i = b_j, \\ \max\{D[i-1, j], D[i, j-1]\}, & a_i \neq b_j. \end{cases}$$

Why the recurrence works. If $a_i = b_j$, some longest common subsequence can end with this common symbol; what precedes it is an LCS of the two shorter prefixes. If $a_i \neq b_j$, a common subsequence cannot use both final symbols, so at least one must be discarded.

The table has $(n + 1)(m + 1)$ entries, each filled in constant time. Thus the running time is $\mathcal{O}(nm)$ and the straightforward space usage is $\mathcal{O}(nm)$.

5.1 Recovering an actual LCS

Start at (n, m) and trace backward:

- if $a_i = b_j$, output that symbol and move to $(i - 1, j - 1)$;
- otherwise move to a neighboring state whose value equals $D[i, j]$.

The symbols are discovered in reverse order. The traceback takes $\mathcal{O}(n + m)$ time once the table is available.

Exercise. Draw the DP table for the two strings in the example and recover one LCS. Where can ties occur, and what do they mean?

DP takeaway. The hardest part is usually not filling the table; it is choosing a state that contains exactly the information needed by the recurrence. Next time we will practice this idea on trees and on the knapsack problem.